

Detecting Network Intrusions Using Snort

Prepared by: Melanie Seguin
Course: Cybersecurity | Fullstack Academy
Date: May 6, 2026

Overview

This lab provides hands-on experience configuring and operating Snort, an open-source intrusion detection system, within a simulated virtual network environment. The objective was to install Snort on an Ubuntu virtual machine, write custom detection rules for two specific traffic types, and verify that those rules triggered correctly when test traffic was generated from a Kali Linux virtual machine. The lab builds foundational skills in signature-based intrusion detection, rule authoring, and real-time log analysis that are directly applicable to Security Operations Center roles.

Situation

Organizations face constant threats from unauthorized access and malicious network activity. Traditional monitoring solutions often lack the flexibility to detect targeted behavior with precision. This project addresses that gap by deploying Snort in a controlled virtual lab to monitor live traffic and trigger alerts based on custom rule configurations. The environment consisted of two Parallels Desktop virtual machines running on an Apple Silicon MacBook: an Ubuntu 24.04 machine acting as the monitored host running Snort, and a Kali Linux 2025.2 machine acting as the traffic source simulating an attacker.

Lab Architecture

Both virtual machines were connected to the same virtual network managed by Parallels Desktop, allowing them to communicate directly with each other. The Ubuntu machine was assigned the IP address 10.211.55.5 and its primary network interface was identified as `enp0s5`. The Kali Linux machine was assigned the IP address 10.211.55.6. Snort was configured to monitor the `enp0s5` interface in promiscuous mode, allowing it to capture all packets traversing the network rather than only those addressed directly to the Ubuntu machine.

Step 1: Ubuntu Virtual Machine Setup

Ubuntu 24.04.3 LTS (ARM64) was provisioned as a new virtual machine through Parallels Desktop on the host MacBook. The system was confirmed operational by booting to the desktop and opening a terminal session. The network interface was verified using the `ip a` command, which confirmed that the interface `enp0s5` was active and had been assigned the IP address 10.211.55.5 on the 10.211.55.0/24 subnet.

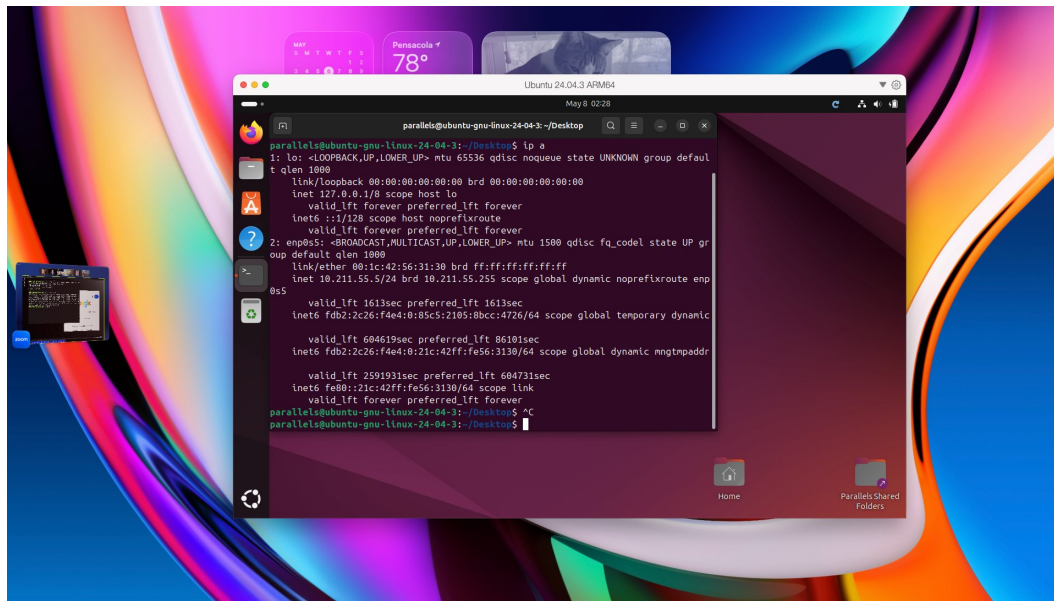


Figure 1: Ubuntu terminal showing the network interface `enp0s5` with IP address 10.211.55.5.

Step 2: Snort Installation and Configuration

Snort version 2.9.20 was installed on the Ubuntu machine using the apt package manager. During installation, the package configuration wizard prompted for the local network range, which was set to 10.211.55.0/24 to match the virtual network subnet. The installation completed successfully and Snort automatically detected the `enp0s5` interface as the default monitoring interface. Installation was verified by running snort with the version flag, which confirmed Snort 2.9.20 GRE Build 82 was active and ready.

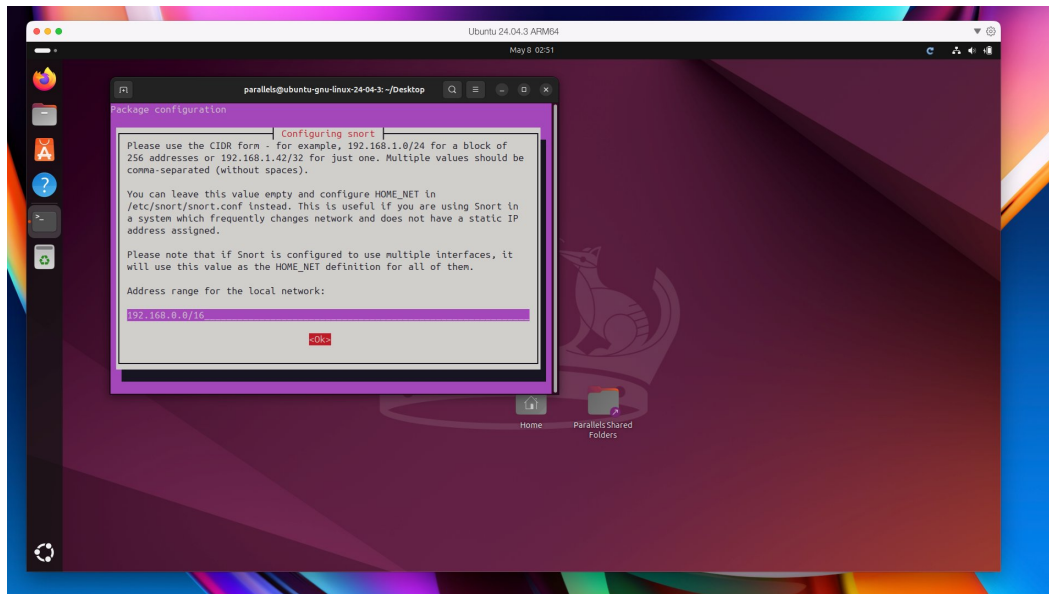


Figure 2: Snort installation prompting for the network range configuration.

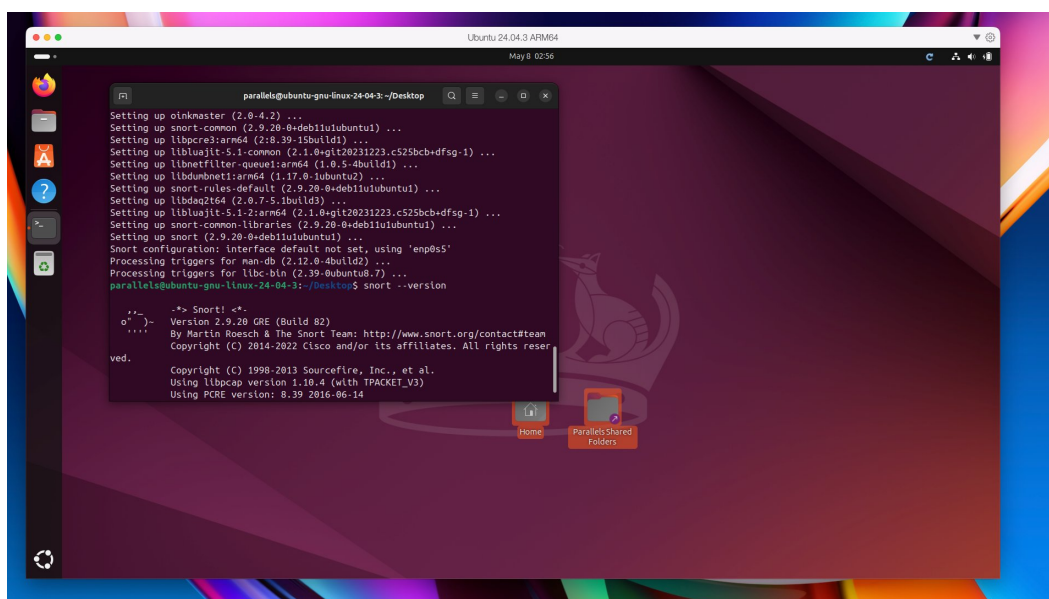


Figure 3: Snort version output confirming successful installation of version 2.9.20.

Step 3: IP Range Configuration

The HOME_NET variable in Snort defines the network range that Snort considers to be the protected internal network. This value was set to 10.211.55.0/24 during installation to match the Parallels virtual network subnet. This configuration ensures that Snort correctly identifies traffic destined for the local network and applies the appropriate detection rules. The custom rules written in the following step reference this HOME_NET variable so that they apply to all hosts on the local subnet.

Step 4: Promiscuous Mode Activation

Promiscuous mode was enabled on the `enp0s5` interface to allow Snort to capture all network packets, not just those addressed directly to the Ubuntu machine. Without promiscuous mode, the network interface would silently discard packets intended for other hosts, limiting Snort's visibility into network activity. The mode was enabled using the `ip link set` command and verified by confirming the `PROMISC` flag appeared in the interface status output.

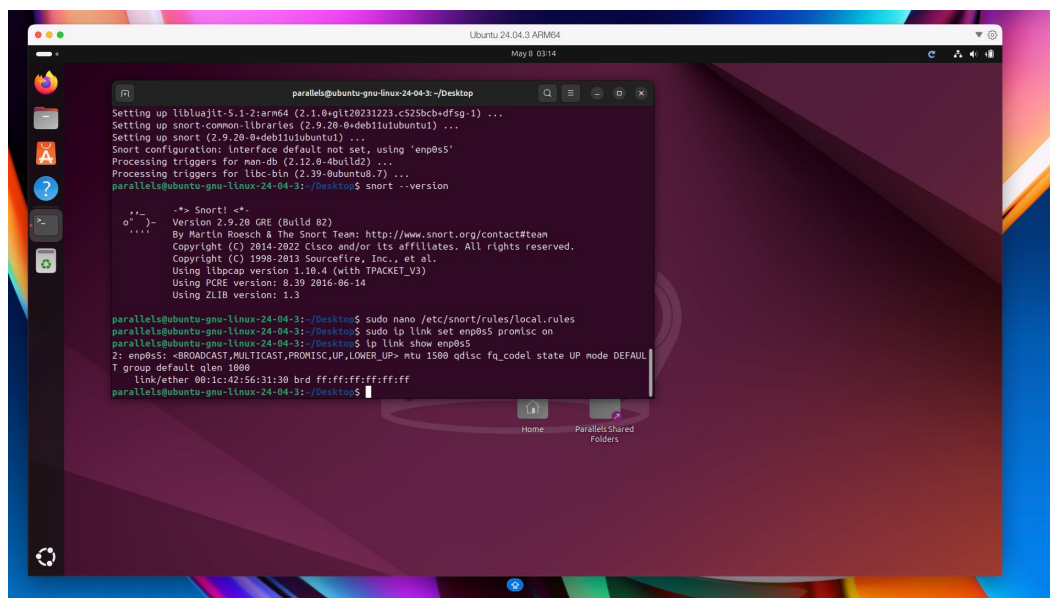


Figure 4: Interface status showing `PROMISC` flag confirming promiscuous mode is active on `enp0s5`.

Step 5: Custom Snort Rule Creation

Two custom detection rules were written in the local rules file located at `/etc/snort/rules/local.rules`. The first rule detects ICMP traffic, which is generated by ping commands used to test network connectivity. The second rule detects TCP traffic on port 22, which is the standard port for SSH remote access connections. Both rules use the `HOME_NET` variable to apply to the protected subnet and are assigned unique signature IDs.

The two rules written were as follows:

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected";
sid:1000001; rev:1;)

alert tcp any any -> $HOME_NET 22 (msg:"SSH Connection Attempt";
sid:1000002; rev:1;)
```

The ICMP rule instructs Snort to generate an alert whenever any ICMP packet arrives at any host on the home network from any source. The SSH rule instructs Snort to generate an alert whenever

a TCP connection is attempted on port 22 of any host on the home network, which indicates an SSH login attempt whether successful or not.

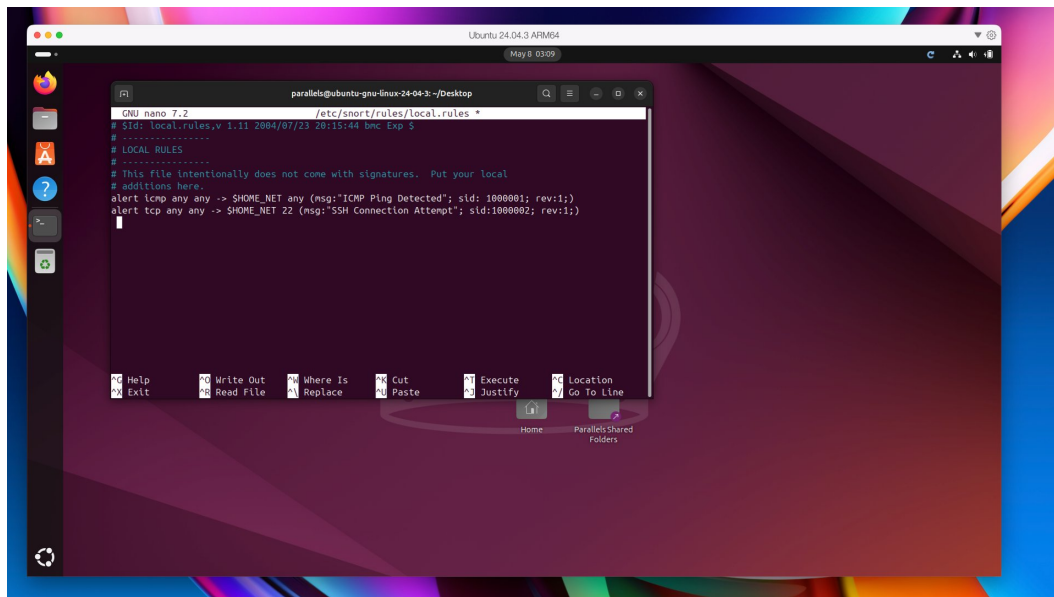


Figure 5: The local.rules file showing both custom Snort detection rules.

Step 6: ICMP Traffic Generation and Log Capture

Snort was started on the Ubuntu machine and configured to monitor the enp0s5 interface using the main snort.conf configuration file. From the Kali Linux machine, a ping command was directed at the Ubuntu machine IP address 10.211.55.5. Kali successfully transmitted 47 ICMP packets with zero packet loss, confirming network connectivity between the two virtual machines. Simultaneously, the Ubuntu terminal displayed a continuous stream of ICMP Ping Detected alerts from Snort, confirming the first custom rule was functioning correctly.

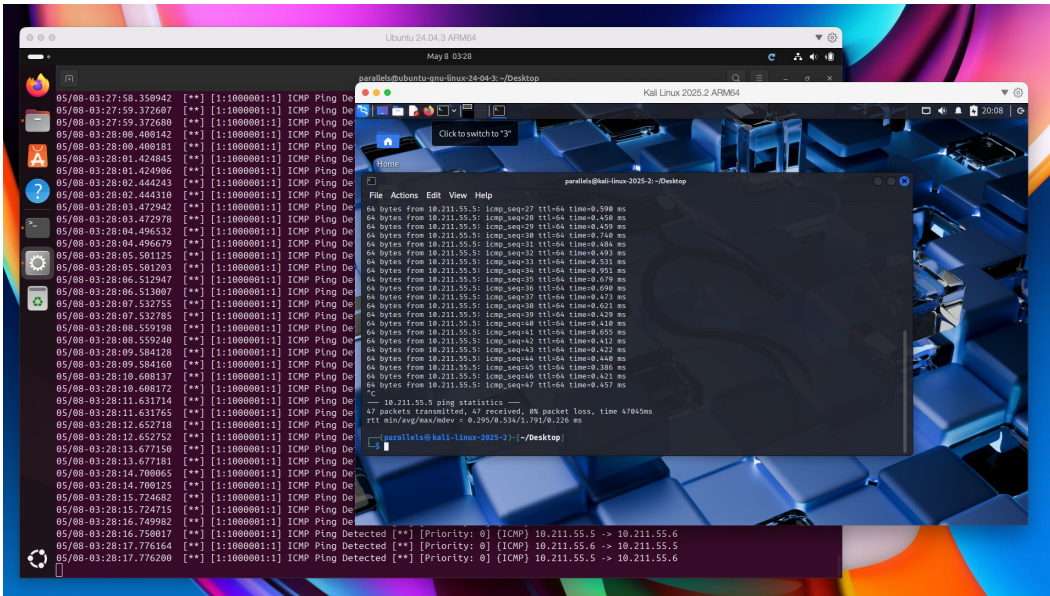


Figure 6: Split view showing Kali Linux ping results alongside Snort ICMP alerts firing on Ubuntu.

Step 7: SSH Traffic Generation and Log Capture

To test the SSH detection rule, the OpenSSH server was installed and started on the Ubuntu machine. From the Kali Linux machine, an SSH connection was initiated targeting the Ubuntu machine at 10.211.55.5 on port 22. The connection was accepted and Kali successfully authenticated and established a remote shell session on the Ubuntu machine, as confirmed by the terminal prompt changing to the Ubuntu hostname. This SSH session generated the traffic needed to trigger the second Snort rule.

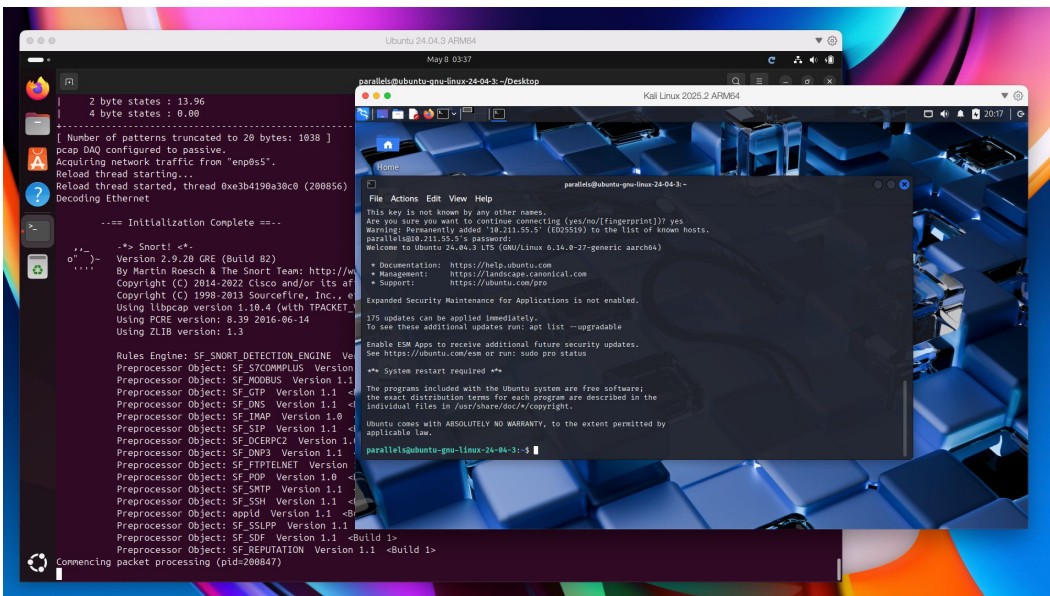


Figure 7: Kali Linux successfully authenticated via SSH and logged into the Ubuntu machine.

Step 8: Rule Testing, Validation and Log Analysis

After stopping Snort, the alert log was examined by reading the `snort.alert.fast` file located in `/var/log/snort/`. The log contained a complete record of all alerts triggered during the session. Both custom rules were confirmed to have fired successfully. The ICMP rule generated alerts with the message ICMP Ping Detected for every ping packet received from Kali. The SSH rule generated alerts with the message SSH Connection Attempt showing the source address 10.211.55.6 connecting to destination 10.211.55.5 on port 22, which precisely identifies the Kali machine initiating the SSH session against the Ubuntu machine.

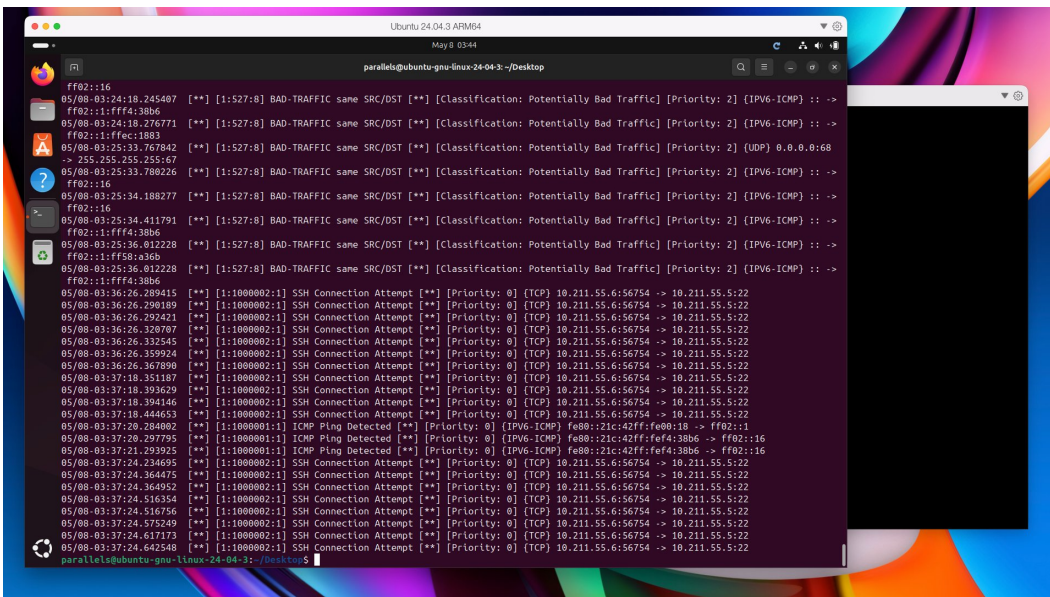


Figure 8: Snort alert log showing both SSH Connection Attempt and ICMP Ping Detected alerts with source and destination IP addresses.

Results and Conclusions

The lab was completed successfully. Snort was installed, configured, and validated as a functional intrusion detection system within a two-machine virtual network environment. Both custom detection rules performed exactly as intended, generating accurate alerts for ICMP and SSH traffic originating from the Kali Linux machine. The alert logs provided clear, timestamped records of each network event including the protocol, source IP, destination IP, and destination port, which is the kind of information a SOC analyst would use to investigate a potential intrusion.

This exercise demonstrated the core workflow of signature-based intrusion detection: define the traffic patterns you want to catch, deploy the sensor, generate test traffic, and verify the alerts match what was expected. The hands-on experience with Snort rule syntax, network interface configuration, and log analysis builds directly applicable skills for network defense and incident

response roles.